# Day – 5 : Numpy Library

### NumPy Library:

NumPy is a core Python library for numerical computing, built for handling large arrays and matrices efficiently. It is significantly faster than Python's built-in lists because it uses optimized C language style storage where actual values are stored at contiguous locations (not object reference).

- **ndarray object**: N-dimensional array for fast numerical operations.
- **Vectorized operations**: Perform element-wise calculations without loops.
- **Broadcasting**: Operate on arrays of different shapes.
- **Linear algebra functions**: Matrix operations like multiplication and inversion.
- **Statistical functions**: Mean, median, standard deviation.
- **Integration**: Works with Pandas, SciPy and scikit-learn.

### NumPy Arrays:

- NumPy arrays (ndarrays) are the backbone of the library.

**Creating Numpy Array**

```
import numpy as np
a = [9,3,3,5]
print(np.array(a))
print(type(a))
```

```
[9 3 3 5]
<class 'list'>
```

**2D matrix Numpy array:**

```
new=np.array([[45,34,22,2],[24,55,3,22]])
print(new)
print(type(new))
```

```
[[45 34 22  2]
 [24 55  3 22]]
<class 'numpy.ndarray'>
```

### dtype:

The desired data-type for the array. If not given, then the type will be determined as the minimum type required to hold the objects in the sequence.

```
n1=np.array([11,23,44] , dtype =float)
n2=np.array([11,23,44] , dtype =bool)
n3=np.array([11,23,44] , dtype =complex)
print(n1)
print(n2)
print(n3)
```

```
[11. 23. 44.]
[ True  True  True]
[11.+0.j 23.+0.j 44.+0.j]
```

**NumPy - Arithmetic Operations**

Arithmetic operations are used for numerical computation and we can perform them on arrays using NumPy. With NumPy we can quickly add, subtract, multiply, divide and get power of elements in an array. NumPy performs these operations even with large amounts of data.

```
import numpy as np
a = np.array([5, 72, 13, 100])
b = np.array([2, 5, 10, 30])
res = np.add(a, b)
print("Addition:",res)
res = np.subtract(a, b)
print("Subtraction:",res)
res = np.multiply(a, b)
print("Multiplication:",res)
res = np.divide(a, b)
print("Division:",res)
res = np.power(a, b)
print("Exponention:",res)
res = np.mod(a, b)
print("Modulus:",res)
```

```
Addition: [  7  77  23 130]
Subtraction: [ 3 67  3 70]
Multiplication: [  10  360  130 3000]
Division: [ 2.5        14.4         1.3         3.33333333]
Exponention: [                25          1934917632        137858491849
 1152921504606846976]
```

## Shape of an Array:

The shape of an array is the number of elements in each dimension.

**Get the Shape of an Array**

NumPy arrays have an attribute called shape that returns a tuple with each index having the number of corresponding elements.

```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
print(arr.shape)
```

```
(2, 4)
```

## Reshaping arrays:

Reshaping means changing the shape of an array.The shape of an array is the number of elements in each dimension.By reshaping we can add or remove dimensions or change number of elements in each dimension.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(4, 3)
print(newarr)
```

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
```

## Iterating Arrays

Iterating means going through elements one by one.As we deal with multi-dimensional arrays in numpy, we can do this using basic for loop of python.If we iterate on a 1-D array it will go through each element one by one.

```
import numpy as np
arr = np.array([1, 2, 3])
for x in arr:
  print(x)
```

```
1
2
3
```

## Joining NumPy Arrays

Joining means putting contents of two or more arrays in a single array.We pass a sequence of arrays that we want to join to the concatenate() function, along with the axis. If axis is not explicitly passed, it is taken as 0.

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.concatenate((arr1, arr2))
print(arr)
```

```
[1 2 3 4 5 6]
```

## Joining Arrays Using Stack Functions

Stacking is same as concatenation, the only difference is that stacking is done along a new axis.We can concatenate two 1-D arrays along the second axis which would result in putting

them one over the other, ie. stacking.We pass a sequence of arrays that we want to join to the stack() method along with the axis. If axis is not explicitly passed it is taken as 0.

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.stack((arr1, arr2), axis=1)
print(arr)
```

```
[[1 4]
 [2 5]
 [3 6]]
```

**Stacking Along Rows**

NumPy provides a helper function: hstack() to stack along rows.

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.hstack((arr1, arr2))
print(arr)
```

```
[1 2 3 4 5 6]
```

**Stacking Along Columns**

NumPy provides a helper function: vstack()  to stack along columns.

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.vstack((arr1, arr2))
print(arr)
```

```
[[1 2 3]
 [4 5 6]]
```

## Splitting NumPy Arrays

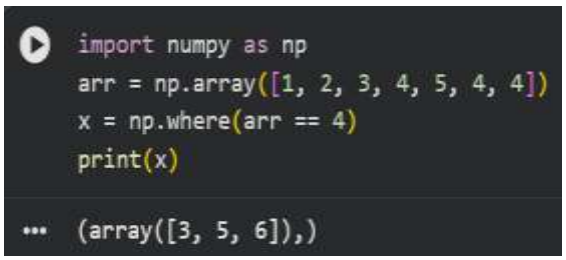
Splitting is reverse operation of Joining.Joining merges multiple arrays into one and Splitting breaks one array into multiple.We use array_split() for splitting arrays, we pass it the array we want to split and the number of splits.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6])
newarr = np.array_split(arr, 3)
print(newarr)
```

```
[array([1, 2]), array([3, 4]), array([5, 6])]
```

## Searching Arrays

You can search an array for a certain value, and return the indexes that get a match.To search an array, use the where() method.

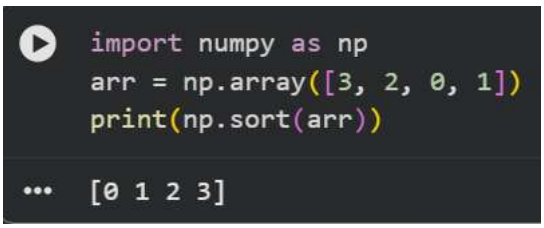


```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 4, 4])
x = np.where(arr == 4)
print(x)
```

```
(array([3, 5, 6]),)
```

## Sorting Arrays

Sorting means putting elements in an ordered sequence.Ordered sequence is any sequence that has an order corresponding to elements, like numeric or alphabetical, ascending or descending.The NumPy ndarray object has a function called sort(), that will sort a specified array.



```
import numpy as np
arr = np.array([3, 2, 0, 1])
print(np.sort(arr))
```

```
[0 1 2 3]
```